

Reliable Multipath and Multisource Content Transmission and Caching for Information-Centric Internet of Things

Xiaonan Wang^{ID}, Yajing Song^{ID}, and Hongbin Cheng

Abstract—Real-world content produced by Internet of Things (IoT) requires real-time transmission to satisfy high-quality surveillance. While information-centric networking (ICN) can achieve rapid content access through in-network caching and aggregation, it does not provide forwarding information on backup providers that perform in-network caching. This limitation leads to content unavailability and single-point failure. Moreover, node resource depletion may cause path disruptions, which disables aggregation and results in content transmission failures. Taking these issues into account, we propose a multipath and multisource content transmission and caching solution. The solution leverages path weights to establish stable multipaths toward optimal providers, sharing content among users. The experimental results justify the feasibility and superiority of the proposed solution.

Index Terms—Information-centric, Internet of Things, multipath, multisource.

I. INTRODUCTION

NOWADAYS, the Internet of Things (IoT) has emerged as a critical means for real-world data acquisition and transmission, particularly for monitoring applications where real-time content delivery is essential to ensure high-quality surveillance [1], [2], [3]. IoT employs the address-based communication model to deliver content, but this end-to-end model cannot achieve rapid content transmission [4]. *First*, content unavailability and single-point failure frequently arises because content is only retrieved from a specific node with limited resources. *Second*, end-to-end transmission is independently performed between a source node and a destination node. In typical IoT applications where n_U users request common content (e.g., environmental data in a specific area), conventional end-to-end transmission proves inefficient as it requires n_U independent delivery processes, leading to considerable content delivery delays and energy consumption through redundant transmissions.

Information-centric networking (ICN) [5] enhances content delivery efficiency through two key mechanisms [6], [7], [8]: 1) in-network caching that reduces content-user distances by leveraging multiple sources, and 2) aggregation that facilitates data sharing among users. Therefore, we integrate ICN

with IoT, called information-centric IoT (ICIoT), to leverage in-network caching and aggregation to enhance content availability and perform rapid content dissemination.

A. Problem and Motivation

As ICN was not originally designed for IoT environments, ICIoT needs to address several challenges to effectively implement in-network caching and aggregation. *First*, in ICN, forwarding information base (FIB) only collects forwarding information on original producers [9], [10], [11], and does not provide forwarding information on backup caching providers. This leads to two drawbacks: 1) suboptimal utilization of in-network caching that could otherwise minimize content-user distances, and 2) content unavailability and single-point failures (SPF). *Second*, IoT nodes have limited energy, and energy depletion induces path disruptions that disable aggregation and cause content transmission failures. *Third*, ICN employs the strategy of caching everything everywhere [12], [13]. That is, content routers cache all received data. IoT nodes have limited storage resources, and this caching strategy is inefficient in IoT environments due to three reasons: 1) Cached content becomes unavailable after its short lifetime; 2) IoT nodes cannot provide cached content after energy depletion; and 3) IoT nodes have to frequently remove old content for new content.

To address these issues, we propose a Low-delay Multipath and Multisource content transmission and caching solution (LMM). To minimize content-user distances through in-network caching and eliminate SPF, the proposal explores multipaths toward multisources, including both original producers and backup providers. To enhance path stability and enable aggregation, the proposal evaluates node resources to construct reliable multipaths toward optimal providers. To enhance content availability and caching efficiency, the proposal assesses both node resources and content features to optimize in-network caching decisions.

B. Novelty and Contribution

Compared to existing solutions, this proposal has the following novelties and contributions.

- 1) A resource allocation-based multipath exploration algorithm is proposed. The algorithm exploits node resources to establish stable multipaths toward multiple sources. These include both original producers and backup providers that implement in-network caching,

Received 16 April 2025; accepted 28 April 2025. Date of publication 7 May 2025; date of current version 9 July 2025. (Corresponding authors: Xiaonan Wang; Hongbin Cheng.)

The authors are with the Department of Computer Science and Engineering, Suzhou University of Technology, Suzhou 215500, China (e-mail: ninawang9@163.com; hbcheng2025@163.com).

Digital Object Identifier 10.1109/IJOT.2025.3566058

thereby mitigating path disruptions caused by resource insufficiency.

- 2) A path weight-based multipath selection algorithm is proposed. The algorithm defines path weights to choose stable paths toward optimal providers, enhancing path stability and facilitating efficient data aggregation.
- 3) An availability-improved multipath content sharing and caching algorithm is proposed. The algorithm utilizes node resources and content features to implement in-network caching, thereby improving content availability and reducing distances between content and users. By establishing stable paths for data aggregation, the algorithm enables efficient content sharing among users from optimal providers, mitigating delivery delays and transmission failures caused by SPF and resource constraints.

In the remainder of this article, Section II discusses the related work on address-centric and ICIoT, LMM is presented and evaluated from Section III to Section VII, respectively, and Section VIII concludes this article.

II. RELATED WORK

In this section, the related work on address-centric IoT and ICIoT is discussed.

A. Address-Centric IoT

Chanak and Banerjee [14] applied IoT to smart healthcare, and present a healthcare content delivery solution. The solution employs a priority-based scheduling strategy to control congestion and enhance delivery ratios. In [15], an address-centric content transmission method is proposed. The method first discovers network topology and uses multidimensional metrics to evaluate node reliability and establish stable routing paths for content delivery. Wang and Chen [16] presented an IoT data transmission method. The method leverages learning automata to probe routing paths, continuously disseminating data without requests. Saxena et al. [17] introduced edge computing to IoT, addressing resource limitations and minimizing end-to-end transmission delays. The solution leverages the resources of edge devices to help transmit data, and the numerical results demonstrate transmission efficiency. Jha et al. [18] presented an end-to-end IoT dissemination method. The method adopts clustering techniques to construct IoT, and uses energy levels and neighbor metrics to select cluster heads, aiming to reduce energy consumption and data access delays. Achour [19] presented an anycast-based content transmission method. The method employs anycast to deliver content between users and optimal anycast members, aiming to reduce content response time. In the method, each user fetches content independently from an optimal anycast member. If the member is incapable of providing content, the content transmission fails.

The above methods have demonstrated that the address-centric mechanism can perform end-to-end content delivery for IoT. However, in IoT applications where multiple users request the same content, address-centric end-to-end delivery proves inefficient, leading to significant energy consumption and delays. Additionally, these methods suffer from content

unavailability, as data can only be retrieved from a specific node.

B. Information-Centric IoT

The ICN standard [5] defines FIB, pending interest table (PIT) and content store (CS) to perform content transmission. Users send *Interest* with a name to fetch content. If nodes receiving *Interest* have target content in CS, they send *Data* back to users. If *Interest* hits their PIT, they wait for *Data*. If *Interest* hits neither CS nor PIT, reverse paths are built by creating PIT and *Interest* is forwarded toward producers based on FIB. During the transmissions of *Data*, nodes along reverse paths cache *Data* in their CS to serve future requests [20], [21].

Araújo et al. [22] utilized ICN to mitigate request flooding and minimize transmission energy consumption. The method implements multicriteria traffic monitoring to detect flooding attacks and filter malicious requests. Saha et al. [23] deployed ICN in IoT, aiming to improve transmission efficiency. The proposed method integrates push-based and pull-based transmission strategies while leveraging broadcast communication for IoT data delivery, thereby enhancing delivery ratios. Wang and Lu [9] leveraged ICN to access rapidly emergency data. The method probes optimal paths by employing node characteristics, facilitating efficient data sharing among consumers. Arsalan et al. [24] combined ICN with IoT and present a content transmission solution to optimize content delivery efficiency. The solution exploits FIB to deliver content, but it does not address the FIB establishment issue. Li et al. [25] presented an underwater content transmission solution. The solution integrates ICN with underwater networks to minimize the distance between users and underwater data, thereby reducing content delivery latency. The solution extends ICN by incorporating a push-based delivery model with in-network caching capabilities. Ud Din et al. [26] proposed the concept of the neighbor FIB (NFIB) to transmit content. Each node utilizes NFIB to forward *Interest* toward potential content providers. Upon receiving *Data*, nodes store it, without evaluating content characteristics.

These works have shown that the in-network caching and aggregation can help enhance the efficiency of IoT-based content delivery. However, these works have the following limitations: 1) FIB does not reflect forwarding information on backup caching providers, leading to content unavailability and SPF; 2) Resource deficiency can cause path disruptions, preventing aggregation and leading to transmission failures; and 3) The ubiquitous caching strategy proves inefficient when resource-constrained nodes store transient content. Taking these limitations into account, we propose a multipath and multisource content transmission and caching for ICIoT, and aim to reduce content transmission delays and failures. The comparisons between the proposal and existing solutions are shown in Table I.

III. SYSTEM MODEL

As shown in Fig. 1, LMM is comprised of three algorithms, namely, path exploration, path selection, and multipath

TABLE I
COMPARISONS

Solution	Ref.	Key features of solutions deployed in UIoT	Issues
Address-centric IoT	[14]-[19]	(1) Each node uses unicast to retrieve data independently. (2) Only the source can access data, and only the destination can provide data.	Single-point failure and high energy consumption and delays under multi-user data retrieval scenarios.
Information-centric IoT	[5], [20]-[26]	(1) In-network caching shortens distances between users and data. (2) Aggregation enables data sharing among users	Inability to provide forwarding information on caching nodes, transmission failures caused by path disruptions, and caching inefficiency caused resource depletion
Proposal	-	(1) Exploit node resources and content features to implement in-network caching, and provide forwarding information on both original producers and backup providers to retrieve data from optimal backup nodes, reducing energy consumption and delays. (2) Utilize path weights to choose stable paths towards optimal providers, mitigating transmission failures caused by path disruptions.	-

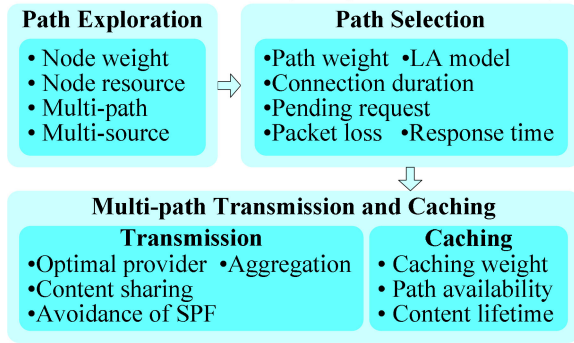


Fig. 1. System model.

transmission and caching. The main functions of the path exploration algorithm are twofold: 1) Define node weights using node resources; and 2) Exploit node weights to collect forwarding information on multisources, including original producers and backup caching providers. The main functions of the path selection algorithm are twofold: 1) Define path weights using path metrics; and 2) Leverage path weights to choose optimal paths toward optimal providers. Path metrics include path connection durations, pending requests, packet loss and content response time. The main functions of the multipath transmission and caching algorithm are threefold: 1) Calculate caching weights using path availability and content lifetime; 2) Perform in-network caching using caching weights; and 3) Leverage optimal paths to perform aggregation, sharing content among users from optimal providers and preventing content transmission failures caused by SPF.

IV. MULTIPATH EXPLORATION USING NODE WEIGHTS

IoT is a multihop network and is modeled as an undirected graph $G = (N, L)$, where N is the node set and L is the internode link set. The nodes that connect with smart user devices, such as laptops or smart phones are called the edge nodes. A catalog of name, denoted as $\{a_1, \dots, a_X\}$, define a catalog of content, denoted as $\{c_1, \dots, c_X\}$.

A. Node Weight

In LMM, node weights are defined to explore multipaths toward multisources. The node weight $w(i)$ of node SN_i is

defined by (1), where $RS(i)$ is the resource set of node SN_i , and $w'(i, j)$ is the weight of the j th resource in $RS(i)$ and is defined by (2)

$$w(i) = \prod_{j=1}^{|RS(i)|} w'(i, j) \quad (1)$$

In (2), $v(i, j)$ is the remaining value of the j th resource in $RS(i)$, and $RT(i, j)$ is the threshold value of the j th resource in $RS(i)$ that is required for caching and transmitting data. Node SN_i conducts caching and transmission operations only if $w(i)$ is greater than zero

$$w'(i, j) = \begin{cases} v(i, j) \cdot RT(i, j)^{-1}, & \text{if } v(i, j) > RT(i, j) \\ 0, & \text{otherwise} \end{cases} \quad (2)$$

B. Exploration of Multipaths Toward Multisources

Each edge node maintains a path exploration table, where each entry includes a name, a node ID set defining a path, a path weight and a lifetime value. Edge node EN_j explores multipaths toward multisources by performing Algorithm-1. As shown in subalgorithm-1, edge node EN_j sends *Interest* with name a_k and node ID set s_k toward coordinates CO_k where content c_k defined by a_k is produced, and then starts timer TM_j (lines 1–3). The initial value of set s_k includes the node ID of node EN_j . During timer TM_j , for each received *Data* with name a_k and set s_k , node EN_j creates a path entry (lines 4–9). As shown in subalgorithm-2, if the distance $d(i, k)$ between coordinates CO_k and node SN_i receiving *Interest* is equal to or greater than distance $d(p, k)$ between CO_k and node SN_p identified by the last node ID in s_k , subalgorithm-2 terminates (lines 1–4). Node SN_i adds its node ID NID_i in set s_k as the last element (line 5). If node SN_i can provide content c_k , it returns *Data* with s_k and a_k to EN_j along the path specified by s_k (lines 6 and 7). Otherwise, if the node weight w_i of node SN_i is greater than zero, it forwards *Interest* (lines 8–11).

Fig. 2 shows the path exploration process. Edge node EN_0 sends *Interest* toward node SN_3 that produces content c_1 defined by name a_1 . After nodes SN_3 , SN_2 , and $SN_{2'}$ which cache content c_1 receive *Interest*, they return *Data* with c_1 to EN_0 through nodes $SN_{2'}$, SN_1 , $SN_{1'}$ and $SN_{1''}$, respectively.

Algorithm 1 Multipath Exploration

Subalgorithm-1 EN_j **starts** **path exploration**

```

1: Begin
2:   Send Interest with  $a_k, s_k$ 
3:   Start TMj
4:   Do
5:     If Receive Data with  $a_k, s_k$ 
6:       Create path entry for  $a_k$  and  $s_k$ 
7:     End If
8:   Until (TMj = 0)
9: End

```

Subalgorithm-2 SN_i receives *Interest* with a_k, s_k

```

1: Begin
2: If  $d(i, k) \geq d(p, k)$ 
3:   Return
4: End If
5:  $s_k = s_k \cup \{NID_i\}$ 
6: If  $c_k$  is TRUE
7:   Send Data with  $a_{kp}, s_k$ 
8: Else If  $w_i > 0$ 
9:   Forward Interest
10: End If
11: End

```

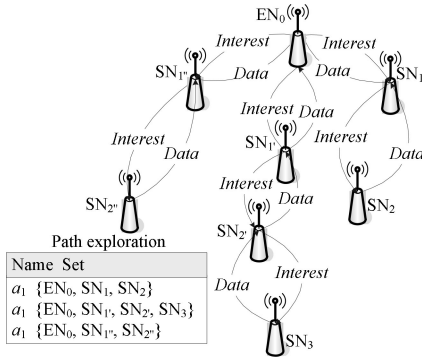


Fig. 2. Path exploration.

Based on the received *Data*, EN_0 creates its path exploration table.

V. MULTIPATH SELECTION USING PATH WEIGHTS

A. Path Weight

The path weigh $p_t(a_k, s_k)$ in a path exploration entry at the time t is defined by (3), where a_k and s_k are the name and node ID set in the entry, respectively. In (3), $d_t(a_k, s_k)$ is the normalized connection duration of the path defined by the node ID set s_k toward the providers caching the content defined by the name a_k at the time t , $q_t(a_k, s_k)$ is the normalized pending requests of the path defined by the node ID set s_k toward the providers caching the content defined by the name a_k , $l_t(a_k, s_k)$ is the normalized packet loss of the path defined by s_k toward the providers caching the content defined by the name a_k , and $o_t(a_k, s_k)$ is the normalized response time of the content

defined by a_k through the path defined by s_k

$$p_t(a_k, s_k) = \frac{d_t(a_k, s_k)}{q_t(a_k, s_k) + l_t(a_k, s_k) + o_t(a_k, s_k)}. \quad (3)$$

The normalized connection duration $d_t(a_k, s_k)$ is defined by (4) and (5), where n_k is the number of paths toward the providers caching content c_k , $d'_t(a_k, s_k)/d'_t(a_k, s'_k)$ is the average connection duration of the path defined by node ID set s_k/s'_k , and $d''_t(m, s_k)$ is the active duration of the m th node within the path defined by s_k

$$d_t(a_k, s_k) = \frac{d'_t(a_k, s_k)}{\text{MAX}_{k' \in [1, n_k]} d'_t(a_k, s_{k'}) + \text{MIN}_{k' \in [1, n_k]} d'_t(a_k, s_{k'})} \quad (4)$$

$$d'_t(a_k, s_k) = |s_k|^{-1} \cdot \sum_{m=1}^{|s_k|} d''_t(m, s_k). \quad (5)$$

The normalized pending requests $q_t(a_k, s_k)$ is defined by (6) and (7), where $q'_t(a_k, s_k)/q'_t(a_k, s'_k)$ is the average pending requests for a_k over the path defined by s_k/s'_k at the time t , and $q''_t(m, s_k)$ is the pending requests for name a_k at the m th node along the path defined by s_k

$$q_t(a_k, s_k) = \frac{q'_t(a_k, s_k)}{\text{MAX}\{1, \text{MAX}_{k' \in [1, n_k]} q'_t(a_k, s_{k'})\} + \text{MIN}_{k' \in [1, n_k]} q'_t(a_k, s_{k'})} \quad (6)$$

$$q'_t(a_k, s_k) = |s_k|^{-1} \cdot \sum_{m=1}^{|s_k|} q''_t(m, s_k). \quad (7)$$

The normalized packet loss $l_t(a_k, s_k)$ is defined by (8) and (9), where $l'_t(a_k, s_k)/l'_t(a_k, s'_k)$ is the packet loss rate for a_k over the path defined by s_k/s'_k at the time t , and $l''_t(m, s_k)$ is the packet loss rate for a_k at the m th node along the path defined by s_k

$$l_t(a_k, s_k) = \frac{l'_t(a_k, s_k)}{\text{MAX}\{1, \text{MAX}_{k' \in [1, n_k]} l'_t(a_k, s_{k'})\} + \text{MIN}_{k' \in [1, n_k]} l'_t(a_k, s_{k'})} \quad (8)$$

$$l'_t(a_k, s_k) = |s_k|^{-1} \cdot \sum_{m=1}^{|s_k|} l''_t(m, s_k). \quad (9)$$

The normalized response time $o_t(a_k, s_k)$ is defined by (10) and (11), where $o'_t(a_k, s_k)/o'_t(a_k, s'_k)$ is the response time for content c_k over the path defined by s_k/s'_k at the time t , and $o''_t(m, s_k)$ is the response time for content c_k at the m th node along the path defined by s_k

$$o_t(a_k, s_k) = \frac{o'_t(a_k, s_k)}{\text{MAX}\{1, \text{MAX}_{k' \in [1, n_k]} o'_t(a_k, s_{k'})\} + \text{MIN}_{k' \in [1, n_k]} o'_t(a_k, s_{k'})} \quad (10)$$

$$o'_t(a_k, s_k) = |s_k|^{-1} \cdot \sum_{m=1}^{|s_k|} o''_t(m, s_k). \quad (11)$$

At the time $t + 1$, the path weight $p_{t+1}(a_k, s_k)$ of the path defined by node ID set s_k is defined by (12), where $p_t(a_k, s_k)$ is the path weight of the path defined by s_k at the time t , λ is

Algorithm 2 EN_j Selects Multipaths

```

1: Begin
2: For  $k = 1$  to  $|PET_j|$ 
3:   Calculate  $p(a_k, s_k)$ 
4: End For
5: For  $x = 1$  to  $n_k$ 
6:   Construct  $ES_k(x)$ 
7:    $\arg \max_{p \in ES_k(x)} p(a_k, s_k) \rightarrow EE'_k(x)$ 
8:   Create  $PE_k(x)$ 
9: End For
10: For  $x = 1$  to  $n_k$ 
11:   If  $(p_k(x) < TP_k)$ 
12:     Remove  $PE_k(x)$ 
13:   End If
14: End For
15: For  $x = 1$  to  $|PT_j| - 1$ 
16:   For  $x' = x + 1$  to  $|PT_j|$ 
17:     If  $(s_x \cap s_{x'} \neq \emptyset)$ 
18:       If  $p(a_k, s_x) > p(a_k, s_{x'})$ 
19:         Remove  $PE_k(x')$ 
20:       Else
21:         Remove  $PE_k(x)$ 
22:       End If
23:     End If
24:   End For
25: End For
26: End

```

an adjustment coefficient, and $r_{t+1}(a_k, s_k)$ is the reward of receiving content c_k through the path defined by s_k

$$p_{t+1}(a_k, s_k) = (1 - \lambda)p_t(a_k, s_k) + \lambda(r_{t+1}(a_k, s_k) + p_t(a_k, s_k)). \quad (12)$$

The reward $r_{t+1}(a_k, s_k)$ is defined by (13), where $d_{t+1}(a_k, s_k)$ is the normalized connection duration of the path defined by s_k toward content c_k at the time $t + 1$, $q_{t+1}(a_k, s_k)$ is the normalized pending requests for a_k over the path defined by s_k at the time $t + 1$, $l_{t+1}(a_k, s_k)$ is the normalized packet loss rate over the path defined by s_k toward content c_k , and $o_{t+1}(a_k, s_k)$ is the normalized response time for content c_k through the path defined by s_k

$$r_{t+1}(a_k, s_k) = \begin{cases} \frac{\sigma \cdot d_{t+1}(a_k, s_k)}{q_{t+1}(a_k, s_k) + l_{t+1}(a_k, s_k) + o_{t+1}(a_k, s_k)}, & \text{if } c_k \text{ is received} \\ \frac{-\sigma \cdot d_{t+1}(a_k, s_k)}{q_{t+1}(a_k, s_k) + l_{t+1}(a_k, s_k) + o_{t+1}(a_k, s_k)}, & \text{otherwise.} \end{cases} \quad (13)$$

B. Multipath Selection

Each edge node maintains a path table, where each entry includes a name, a node ID set and a lifetime value. Edge node EN_j with path exploration table PET_j selects multipaths toward optimal multisources with content c_k , and builds its path table PT_j through Algorithm-2.

For each exploration entry EE_k in PET_j , edge node EN_j calculates path weight $p(a_k, s_k)$, where a_k and s_k are the name and node ID set in EE_k , respectively, (lines 1–4). If node EN_j

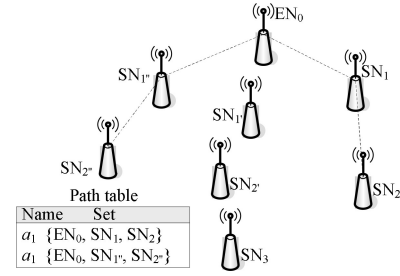


Fig. 3. Path selection.

explores n_k providers with content c_k , for the x th provider $p_k(x)$, node EN_j does the following operations: 1) Construct exploration entry set $ES_k(x)$, where each exploration entry $EE_k(x)$ defines a path toward $p_k(x)$; 2) Choose exploration entry $EE'_k(x)$ with the maximal path weight; and 3) Create path entry $PE_k(x)$, where the name and node ID set are equal to those in $EE'_k(x)$ (lines 5–9). If path weight $p_k(x)$ of path entry $PE_k(x)$ is smaller than threshold TP_k , $PE_k(x)$ is removed (lines 10–14). For any two different path entries $PE_k(x)$ with node ID set s_x and $PE_k(x')$ with node ID set $s_{x'}$, if the intersection of s_x and $s_{x'}$ is not equal to $\emptyset$, the entry with the smaller path weight is removed (lines 15–26).

Fig. 3 shows the path selection process. Edge node EN_0 calculates the path weights, selects the optimal paths toward different providers according to their path weights, and creates its path table.

VI. MULTIPATH TRANSMISSIONS AND CACHING**A. Caching Weight**

Edge node EN_i selects path entry set PS_k where the name of each path entry is a_k . For name a_k , the content availability $b(a_k)$ of paths defined by set PS_k is defined by (14), where s_x is the node ID set of the x th path entry in set PS_k

$$b(a_k) = \begin{cases} \sum_{x=1}^{|PS_k|} |s_x|^{-1}, & \text{if } \sum_{x=1}^{|PS_k|} |s_x|^{-1} < TB_k \\ 0, & \text{otherwise.} \end{cases} \quad (14)$$

The normalized lifetime of content c_k defined by a_k is defined by (15), where t_c is the current time, $g(a_k)$ is the time when c_k is generated, $t'(a_k)$ is the lifespan of c_k , TB_k , and TT_k are the threshold values of content availability and content lifetime required by performing in-network caching, respectively

$$f(a_k) = \begin{cases} (t_c - g(a_k)) \cdot t'(a_k)^{-1}, & (t_c - g(a_k)) \cdot t'(a_k)^{-1} \geq TT_k \\ 0, & \text{otherwise.} \end{cases} \quad (15)$$

Caching weight $c(a_k)$ is defined (16)

$$c(a_k) = b(a_k) \cdot f(a_k). \quad (16)$$

If $c(a_k)$ is zero, node EN_i sets its caching node ID cID_k to empty. Otherwise, node EN_i selects caching node $SN_{x'}$ that satisfies (17) and sets its caching node ID cID_k to the node ID of $SN_{x'}$. In (17), n_k is the number of nodes involved in the path toward providers caching c_k , $w(x)$ is the node weight of

the x^{th} node, and $d_x(i)$ is the distance between the x^{th} node and node EN_i

$$\arg \text{MAX}_{x \in [1, n_k]} w(x) \cdot d_x(i) \rightarrow SN_{x'}. \quad (17)$$

B. Multipath Transmission and Caching

User U_i executes Algorithm-3 to acquire content c_k defined by name a_k . As shown in subalgorithm-1, U_i sends *Interest* with a_k to edge node EN_i with path table PT_i , and then starts timer TM_i (lines 1–3). During timer TM_i , if user U_i receives *Data* with c_k , it terminates subalgorithm-1 (lines 4–9). After EN_i receives *Interest* from U_i , it executes subalgorithm-2. For each path entry PE_j in PT_i , node EN_i sets node ID set s_k in *Interest* to node ID set s_j in PE_j and the caching node ID to cID_k , forwards *Interest* and start timer $TM_{i'}$ (lines 1–6). During $TM_{i'}$, for each received *Data*, if EN_i has no path exploration entry EE_k where the last element of the node ID set is equal to the last element in s_j in *Data*, it calculates path weight $p(a_k, s_k)$ and creates a path exploration entry EE_k for a_k , s_j and $p(a_k, s_k)$ (lines 7–11). Otherwise, EN_i calculates reward $r_{t+1}(a_k, s_k)$ where $t+1$ is the time when *Data* is received, computes the path weight $p_{t+1}(a_k, s_k)$ using $r_{t+1}(a_k, s_j)$, and updates its path table PT_i and cID_k (lines 12–19). As shown in subalgorithm-3, if node SN_x can provide c_k , it returns *Data* with a_k , c_k and s_k , and ends subalgorithm-3 (lines 1–5). Node ID set s_k includes the nodes involved in the path from EN_i to SN_x . If SN_x is involved in s_k and has no PIT entry PE_k for a_k , it creates PE_k and forwards *Interest* (lines 6–10). As shown in subalgorithm-4, if node ID NID_y of node SN_y is equal to cID_k , it caches *Data* (lines 1–4). If node SN_y has PIT entry PIE_k for a_k , it forwards *Data* and removes PIE_k (lines 5–9).

Fig. 4 explains the process of users U_1 , U_2 , and U_3 sharing content from optimal providers. As shown in Fig. 4(a), user U_1 sends *Interest* with name a_1 that is forwarded toward optimal providers SN_2 and SN_2'' . During *Interest* transmission, nodes EN_0 , SN_1 , and SN_1'' create PIT entries for name a_1 to build reverse paths. Then, user U_2 sends *Interest* with name a_1 to node SN_1 , as shown in Fig. 4(b). Since node EN_0 has the PIT entry for a_1 , aggregation is implemented. Thus, users U_1 and U_2 share content through aggregation, and node SN_1'' performs in-network caching. After edge node EN_0 receives *Data*, it calculates rewards and path weights to update its path table. As shown in Fig. 4(c), user U_3 sends *Interest* with a_1 that is forwarded toward optimal providers SN_2 and SN_1'' . Since node SN_2 is incapable of providing content c_1 , user U_3 retrieves content c_1 from optimal provider SN_1'' , eliminating SPF, and meanwhile EN_0 updates its path table according to path weights.

VII. EVALUATION

This framework leverages in-network caching and aggregation to establish multipaths toward multisources, aiming to 1) reduce user-content hop distances; 2) improve content access success rates; and 3) minimize response time and energy consumption. In LMM, n_U users share content through two situations. In the first situation (called LMM-1), content is delivered between users and providers. In the second

Algorithm 3 Multipaths Transmission and Caching

Subalgorithm-1 U_i retrieves c_k

```

1: Begin
2:   Send Interest with  $a_k$ 
3:   Start  $TM_i$ 
4:   Do
5:     If Receive Data with  $a_k, c_k$ 
6:       Return
7:     End If
8:   Until ( $TM_i = 0$ )
9: End

```

Sub-algorithm-2 EN_i receives *Interest* with a_k

```

1: Begin
2:   For  $j = 1$  to  $|PT_i|$ 
3:     Set  $s_k$  to  $s_j$ 
4:     Send Interest with  $a_k, s_k, cID_k$ 
5:   End For
6:   Start  $TM_{i'}$ 
7:   Do
8:     If Receive Data with  $a_k, s_k, c_k$ 
9:       If ( $EE_k$  is FALSE)
10:        Calculate  $p(a_k, s_k)$ 
11:        Create  $EE_k$ 
12:       Else
13:        Calculate  $r_{t+1}(a_k, s_k)$ 
14:        Calculate  $p_{t+1}(a_k, s_k)$ 
15:        Update  $PT_i$  and  $cID_k$ 
16:       End if
17:     End if
18:   Until ( $TM_{i'} = 0$ )
19: End

```

Sub-algorithm-3 SN_x receives *Interest* with a_k, s_k

```

1: Begin
2:   If ( $c_k$  is TRUE)
3:     Send Data with  $a_k, c_k, s_k$ 
4:     Return
5:   End if
6:   If ( $(SN_x \in s_k) \text{ AND } (PE_k \text{ is FALSE})$ )
7:     Create  $PE_k$ 
8:     Forward Interest
9:   End if
10: End

```

Sub-algorithm-4 SN_y receives *Data* with a_k, s_k, c_k, cID_k

```

1: Begin
2:   If ( $NID_y = cID_k$ )
3:     Cache Data
4:   End if
5:   If ( $PIE_k$  is TRUE)
6:     Forward Data
7:     Remove  $PIE_k$ 
8:   End if
9: End

```

situation (called LMM-2), content is transmitted between users and intermediate nodes that perform aggregation, called aggregation nodes. If n_P providers are selected to provide

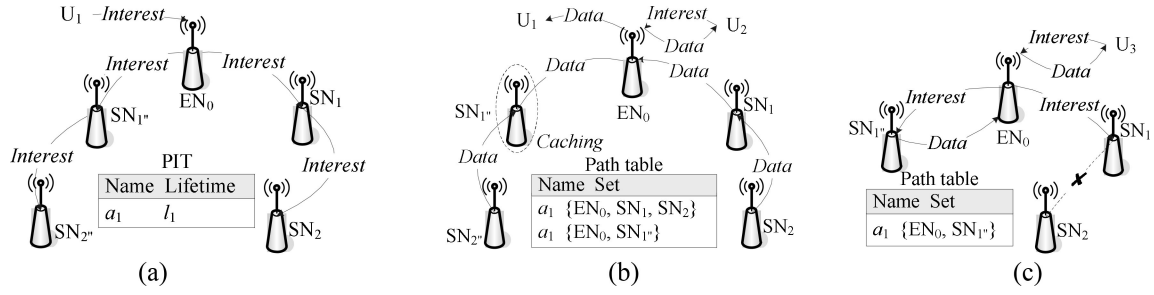


Fig. 4. Multisource transmission and caching. (a) Request forwarding. (b) Caching. (c) Content retrieval.

content, the distance $h_{LMM}(n_U, n_P)$ between n_U users and providers is defined by (18) and (19), where $h(x, n_P)$ is the distance (hop) between the x^{th} user and content, $h'(x, y)$ is the distance between the x^{th} user to the y^{th} provider, n_A is the number of nodes involved in n_P paths, and $h''(x, z)$ is the distance between the x^{th} user to the z^{th} node involved in n_P paths

$$h_{LMM}(n_U, n_P) = n_U^{-1} \cdot \sum_{x=1}^{n_U} h(x, n_P) \quad (18)$$

$$h(x, n_P) = \begin{cases} \min_{y=1}^{n_P} h'(x, y), LMM - 1 \\ \min_{z=1}^{n_A} h''(x, z), LMM - 2, n_A = \sum_{y=1}^{n_P} h'(x, y). \end{cases} \quad (19)$$

The success rate $s_{LMM}(n_U, n_P)$ of sharing content among n_U users is defined by (20) and (21). In (20) and (21), $s(x, n_P)$ is the success rate of the x^{th} user sharing content, $s'(y, z)$ is the forwarding success rate of the z^{th} node involved in the y^{th} path, and $s''(x, z)$ is the forwarding success rate of the z^{th} node along the path from the x^{th} user to its aggregation node

$$s_{LMM}(n_U, n_P) = \prod_{x=1}^{n_U} s(x, n_P) \quad (20)$$

$$s(x, n_P) = \begin{cases} \max_{y=1}^{n_P} \prod_{z=1}^{h'(x,y)} s'(y, z), LMM - 1 \\ \prod_{z=1}^{h(x,n_P)} s''(x, z), LMM - 2. \end{cases} \quad (21)$$

The time $t_{LMM}(n_U, n_P)$ taken by n_U users to share content is defined by (22) and (23). In (22) and (23), $t(x, n_P)$ is the time taken by the x^{th} user to share content, $t_{Int}(y, z)$ and $t_{Data}(y, z)$ are the time taken by the z^{th} node involved in the y^{th} path to transmit *Interest* and *Data*, respectively, and $t'_{Int}(y, z)$ and $t'_{Data}(y, z)$ are the time taken by the z^{th} node to transmit *Interest* and *Data* along the path from the x^{th} user to its aggregation node, respectively

$$t_{LMM}(n_U, n_P) = n_U^{-1} \cdot \sum_{x=1}^{n_U} t(x, n_P) \quad (22)$$

$$t(x, n_P) = \begin{cases} \min_{y=1}^{n_P} \sum_{z=1}^{h'(x,y)} (t_{Int}(y, z) + t_{Data}(y, z), LMM - 1 \\ \sum_{z=1}^{h(x,n_P)} (t'_{Int}(x, z) + t'_{Data}(x, z)), LMM - 2. \end{cases} \quad (23)$$

The energy $e_{LMM}(n_U, n_P)$ consumed by n_U users to share content is defined by (24) and (25). In (24) and (25), $e(x, n_P)$ is the energy consumed by the x^{th} user to share content, $e'_{Int}(y, z)$ and $e'_{Data}(y, z)$ are the energy consumed by the

z^{th} node involved in the y^{th} path to transmit *Interest* and *Data*, respectively, and $e'_{Int}(y, z)$ and $e'_{Data}(y, z)$ are the energy consumed by the z^{th} node to transmit *Interest* and *Data* along the path from the x^{th} user to its aggregation node, respectively

$$e_{LMM}(n_U, n_P) = n_U^{-1} \cdot \sum_{x=1}^{n_U} e(x, n_P) \quad (24)$$

$$e(x, n_P) = \begin{cases} \min_{y=1}^{n_P} \sum_{z=1}^{h'(x,y)} (e'_{Int}(y, z) + e'_{Data}(y, z)), LMM - 1 \\ \sum_{z=1}^{h(x,n_P)} (e'_{Int}(x, z) + e'_{Data}(x, z)), LMM - 2. \end{cases} \quad (25)$$

The path exploration algorithm acquires forwarding information about backup providers by probing paths toward the original data producer. Both the forwarding information learning process and path establishment procedure are independent of the network size, the complexity of the algorithm is $O(1)$. The path selection algorithm determines optimal paths through local learning of path parameters regardless of the network size, so the complexity of the algorithm is $O(1)$. In the transmission and caching algorithm, nodes locally cache data piggybacked in *Data* while users share data from optimal backup providers. As backup providers are distributed across the network and are positioned near users, the distance between users and optimal providers remains unaffected by the network size and thus the complexity of the algorithm is $O(1)$. Consequently, these algorithms are scalable to large IoT environments.

The proposal is evaluated in *ns-3*. According to [3], [9], and [20], the IoT area is $150 \text{ m} \times 150 \text{ m}$, n_P ranges from 4 to 10, n_C ranges from 2 to 8, the communication range is 20 m, and the number of nodes is 120. The experiment lasts 3600 s and is iterated 50 times. The experiment consists of three stages. At the multipath exploration stage, node weights are used to collect forwarding information on multisources. At the multipath selection stage, path weights are leveraged to choose optimal paths toward providers. In the multipath transmission and caching phase, caching weights are utilized to perform in-network caching, and optimal paths are employed to perform aggregation, achieving rapid content sharing among users. LMM-1, LMM-2, and LMM are compared to observe the effects of in-networking caching and aggregation. Fig. 5 demonstrates the effects of n_P on the distance between content and users, content transmission success rates, content response time and energy consumption

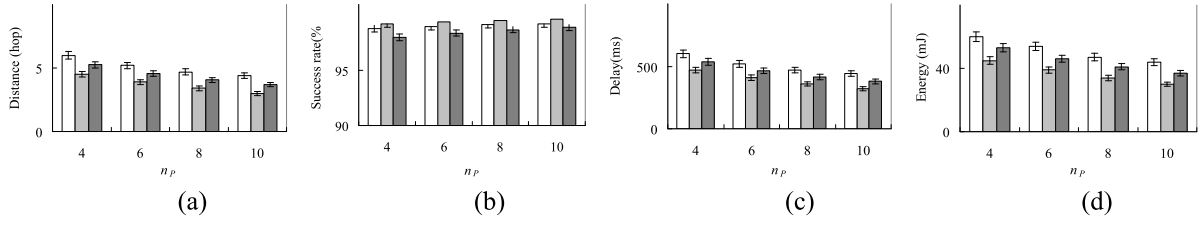


Fig. 5. The effects of n_p . (a) n_p -based distance. (b) n_p -based success rate. (c) n_p -based delay. (d) n_p -based energy.

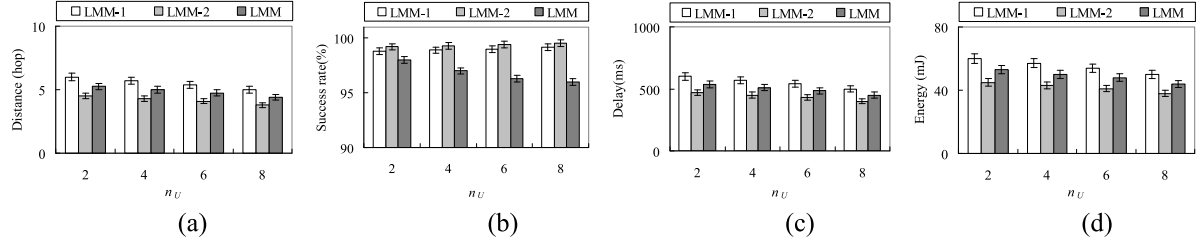


Fig. 6. The effects of n_U . (a) n_U -based distance. (b) n_U -based success rate. (c) n_U -based delay. (d) n_U -based energy.

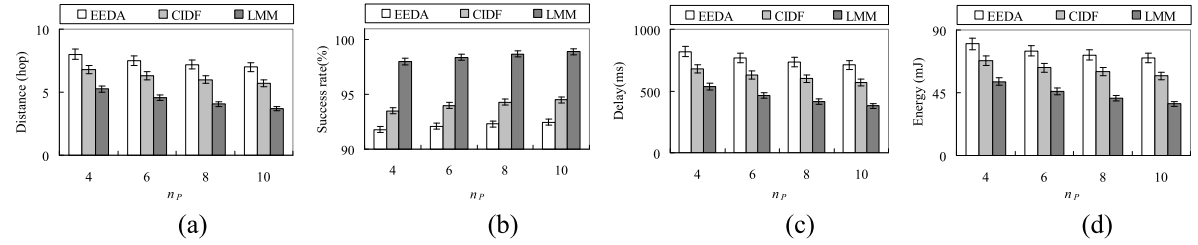


Fig. 7. Comparisons based on n_p . (a) n_p -based distance comparisons. (b) n_p -based success rate comparisons. (c) n_p -based delay comparisons. (d) n_p -based energy comparisons.

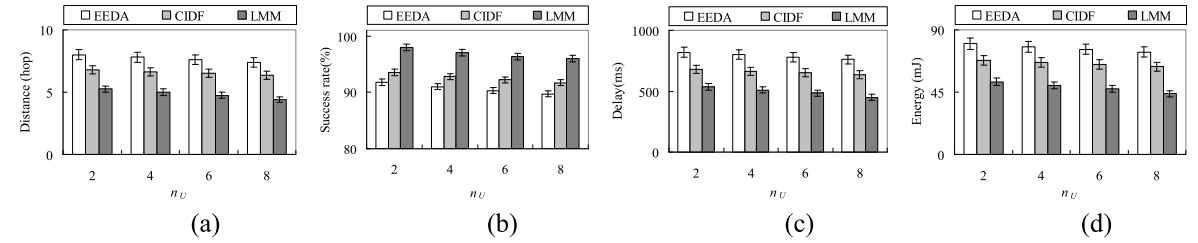


Fig. 8. Comparisons based on n_U . (a) n_U -based distance comparisons. (b) n_U -based success rate comparisons. (c) n_U -based delay comparisons. (d) n_U -based energy comparisons.

in LMM-1, LMM-2, and LMM. The growth in n_p shortens the distance between users and optimal providers, so the hop count between users and content is reduced. This reduces both the probability of path disruptions and content delivery latency, thereby improving content transmission success rates while decreasing response time and energy consumption, as shown in Fig. 5(a)–(d). Fig. 6 shows the effects of n_U on LMM-1, LMM-2, and LMM. The increase in n_U reduces the content-user distance, as demonstrated in Fig. 6(a)–(d), which in turn: 1) decreases the probability of path interruptions, enhancing content transmission success rates; 2) reduces content delivery latency; and 3) lowers energy consumption. Figs. 5 and 6 demonstrate that LMM-1 exhibits maximal values for content-user distance, response time, and energy consumption, while LMM-2 achieves minimal values for these metrics. This difference stems from their distinct delivery

approaches. LMM-1 transmits content between users and optimal providers, whereas LMM-2 delivers data between users and aggregation nodes along paths toward optimal providers. LMM-2 achieves maximal content delivery success rates compared to LMM-1's minimal rates, primarily because LMM-2 minimizes content-user distances, thereby improving path stability.

LMM is compared to EEDA [9] and CIDF [19]. Fig. 7 demonstrates the effects of n_p on EEDA, CIDF, and LMM in terms of content-user distances, success rates, data access delays and energy consumption. The increase in n_p reduces the distance between users and content, thereby decreasing both content response time and energy consumption, as shown in Fig. 7(a)–(c). This distance reduction simultaneously lowers the probability of path disruptions, resulting in improved transmission success rates, as shown in Fig. 7(d). Fig. 8 compares

how n_U affects EEDA, CIDF, and LMM. As n_U grows, the content-user distance decreases. This enhances success rates, reduces data response time, and lowers energy consumption, as shown in Fig. 8(a)–(d).

Figs. 7 and 8 demonstrate that LMM achieves simultaneously minimal content-user distances, response time and energy consumption, while attaining maximal success rates. This superior performance stems from two key architectural advantages. First, LMM employs a weighted multipath mechanism that: 1) utilizes node weights to acquire forwarding information from multiple sources and 2) leverages path weights to select optimal, stable paths, enabling aggregation and avoiding transmission failures caused by both SPF and resource exhaustion. In contrast, CIDF and EEDA are unaware of forwarding information on caching providers. Second, LMM exploits caching weights to optimize content caching, reducing content-user distances. Moreover, LMM selects optimal paths to enable aggregation, achieving rapid and energy-consumption content sharing. While CIDF supports aggregation, its performance is limited by two critical shortcomings: 1) resource deficiency frequently causes reverse-path disruptions, leading to both aggregation failures and content delivery failures and 2) content caching operates without considering content characteristics. EEDA implements neither aggregation nor in-network caching, requiring each user to independently retrieve content.

The proposed approach implements in-network caching by harnessing node resources and content features, distributing backup providers across the IoT network. As a result, the distance between users and optimal backup providers remains unaffected by the network size, since these providers are positioned near users. By dynamically learning real-time forwarding information on backup providers, the approach ensures efficient data retrieval from optimal providers and is hardly impacted by the network size, thereby enabling scalability in large IoT environments.

VIII. CONCLUSION

This proposal presents a multipath and multisource content transmission and caching solution for ICIoT. At the current stage, we implement the proposal in the experimental environment where the network size is fixed and content is delivered in plaintext, and aim to: 1) validate the feasibility and effectiveness of the proposal; 2) observe whether multipaths toward multisources can help improve content delivery success rates; 3) observe whether stable paths can help reduce content delivery delays and energy consumption; and 4) observe whether forwarding information on backup providers can help shorten content-user distances, alleviate content delivery delays and energy consumption, and enhance success rates.

In real-world IoT applications, the network size may vary and thousands of nodes may be involved. Moreover, plaintext transmission poses security risks. Encryption can address security concerns and handle malicious nodes, preventing data tampering and leakage. That is, before transmission, data is encrypted and sent as ciphertext over the network. Even if malicious nodes intercept the ciphertext, they cannot decrypt it. This ensures protection against data tampering and leakage.

In future work, we will implement the proposed framework in large-scale real-world network environments with varying scales, and employ encryption to safeguard data security.

REFERENCES

- [1] M. A. Naeem, I. U. Din, Y. Meng, A. Almogren, and J. J. P. C. Rodrigues, "Centrality-based on-path caching strategies in NDN-Based Internet of Things: A survey," *IEEE Commun. Surveys Tuts.*, early access, Nov. 7, 2024, doi: [10.1109/COMST.2024.3493626](https://doi.org/10.1109/COMST.2024.3493626).
- [2] D. Gao et al., "Physical-layer CTC From BLE to Wi-Fi with IEEE 802.11ax," *IEEE Trans. Mobile Comput.*, vol. 24, no. 1, pp. 338–351, Jan. 2025.
- [3] X. Wang and X. You, "Efficient data sharing and caching for information-centric IoT," *IEEE Internet Things J.*, vol. 11, no. 10, pp. 18074–18081, May 2024.
- [4] W.-K. Jia, L.-T. Chen, and Z.-X. Xu, "An end-to-end IP header compressed packet forwarding framework for bandwidth-constrained networks," *IEEE Trans. Green Commun. Netw.*, vol. 6, no. 4, pp. 2156–2167, Dec. 2022.
- [5] B. Wisingh, C. Wood, A. Afanasyev, L. Zhang, S. Oran, and X. Tschudin, "Information-centric networking (ICN): Content-centric networking (CCNx) and named data networking (NDN) terminology," Internet Eng. Task Force, RFC 8793, 2020.
- [6] M. A. Naeem, T. N. Nguyen, R. Ali, K. Cengiz, Y. Meng, and T. Khurshaid, "Hybrid cache management in IoT-based named data networking," *IEEE Internet Things J.*, vol. 9, no. 10, pp. 7140–7150, May 2022.
- [7] M. M. Karim, K. Sharif, S. Biswas, Z. Latif, Q. Qu, and F. Li, "CIC-SIoT: Clean-slate information-centric software-defined content discovery and distribution for Internet of Things," *IEEE Internet Things J.*, vol. 11, no. 22, pp. 37140–37153, Nov. 2024.
- [8] H. Bai et al., "QSCCP: A QoS-aware congestion control protocol for information-centric networking," *IEEE Trans. Netw. Service Manag.*, vol. 22, no. 1, pp. 532–556, Feb. 2025.
- [9] X. Wang and Y. Lu, "Information-centric robotic ad hoc networking based continuous data routing and delivery for disaster scenes," *IEEE Trans. Green Commun. Netw.*, early access, Sep. 25, 2024, doi: [10.1109/TGCN.2024.3467267](https://doi.org/10.1109/TGCN.2024.3467267).
- [10] M. Imran, M. N. Ali, M. S. U. Din, M. A. U. Rehman, and B.-S. Kim, "An efficient communication and computation resources sharing in information-centric 6G networks," *IEEE Internet Things J.*, vol. 11, no. 16, pp. 27275–27294, Aug. 2024.
- [11] X. Wang and R. Zhang, "Interest flooding suppression-based routing for information-centric Internet of Robotic Things," *IEEE Internet Things J.*, vol. 11, no. 19, pp. 32197–32204, Oct. 2024.
- [12] Z. Zhang, X. Wei, C.-H. Lung, and Y. Zhao, "iCache: An intelligent caching scheme for dynamic network environments in ICN-based IoT networks," *IEEE Internet Things J.*, vol. 10, no. 2, pp. 1787–1799, Jan. 2023.
- [13] Q. Wang, B. Lee, N. Murray, and Y. Qiao, "ECE: Exactly once computation for collaborative edge in IoT using information-centric networking," *IEEE Internet Things J.*, vol. 10, no. 19, pp. 17338–17355, Oct. 2023.
- [14] P. Chanak and I. Banerjee, "Congestion free routing mechanism for IoT-enabled wireless sensor networks for smart healthcare applications," *IEEE Trans. Consum. Electron.*, vol. 66, no. 3, pp. 223–232, Aug. 2020.
- [15] T. Ali et al., "A secure communication in IoT enabled underwater and wireless sensor network for smart cities," *Sensors*, vol. 20, no. 15, p. 4309, 2020.
- [16] X. Wang and X. Chen, "Learning automata based routing and data dissemination for underwater wireless sensor networks," *IEEE Trans. Green Commun. Netw.*, early access, Feb. 11, 2025, doi: [10.1109/TGCN.2025.3540872](https://doi.org/10.1109/TGCN.2025.3540872).
- [17] M. Saxena, S. Srivastava, V. K. Dwivedi, R. Chitranshi, and P. K. Mishra, "Minimizing end-to-end latency in edge computing-enabled IoT networks through edge-to-edge resource allocation," in *Proc. Int. Conf. IoT, Commun. Autom. Technol. (ICICAT)*, 2024, pp. 1512–1517.
- [18] A. V. Jha, B. Appasani, M. S. Khan, and H. H. Song, "A novel clustering protocol for network lifetime maximization in underwater wireless sensor networks," *IEEE Trans. Green Commun. Netw.*, vol. 8, no. 4, pp. 1371–1384, Dec. 2024, doi: [10.1109/TGCN.2024.3375011](https://doi.org/10.1109/TGCN.2024.3375011).

- [19] M. Achour, "Minimizing end-to-end delay with anycast and controlled mobility in sensor networks," *Wireless Pers. Commun.*, vol. 118, pp. 2621–2630, Feb. 2021.
- [20] X. Wang and Y. Song, "Personalized preference and social attribute-based data sharing for information-centric IoT," *IEEE Trans. Netw. Service Manag.*, vol. 22, no. 2, pp. 1473–1483, Apr. 2025, doi: [10.1109/TNSM.2025.3529291](https://doi.org/10.1109/TNSM.2025.3529291).
- [21] Z. Zhang, C.-H. Lung, X. Wei, M. Chen, S. Chatterjee, and Z. Zhang, "In-network caching for ICN-based IoT (ICN-IoT): A comprehensive survey," *IEEE Internet Things J.*, vol. 10, no. 16, pp. 14595–14620, Aug. 2023.
- [22] F. R. C. Araújo, A. L. R. Madureira, and L. N. Sampaio, "A multicriteria-based forwarding strategy for interest flooding mitigation on named data wireless networking," *IEEE Trans. Mobile Comput.*, vol. 22, no. 12, pp. 7000–7013, Dec. 2023.
- [23] K. K. Saha, S. Ray, and M. Dasgupta, "ECMHP: ECC-based secure handshake protocol for multicasting in CCN-IoT environment," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 5, pp. 5826–5842, Oct. 2024.
- [24] A. Arsalan, M. Burhan, R. A. Rehman, T. Umer, and B.-S. Kim, "E-DRAFT: An efficient data retrieval and forwarding technique for named data network based wireless multimedia sensor networks," *IEEE Access*, vol. 11, pp. 15315–15328, 2023.
- [25] J. Li et al., "Information-centric wireless sensor networking scheme with water-depth-awareness content caching for underwater IoT," *IEEE Internet Things J.*, vol. 9, no. 2, pp. 858–867, Jan. 2022.
- [26] M. S. Ud Din, M. A. Ur Rehman, and B.-S. Kim, "CIDF-WSN: A collaborative interest and data forwarding strategy for named data wireless sensor networks," *Sensors*, vol. 21, no. 15, p. 5174, 2021.